

Assignment: Maze Pathfinding Competition

Objective

Design and optimize a pathfinding algorithm to navigate through a maze with **maximum efficiency**. Students are encouraged to use LLM to research, analyze, and optimize algorithms.

Problem Statement

Write a pathfinding algorithm for **multiple robots to escape from a maze simultaneously** in the **fewest steps possible**.

Technical Constraints

- **Maze Size:** From 20×20 up to 50×50
- **Step Limit:** Maximum of $N \times N \times \text{Bots}$ steps per maze
- **Programming Language:** JavaScript

Assignment Format

- **Team Size:** 3 students per team
- **Submission:** Upload **one** JavaScript file containing the `MazeFinder` class to Moodle by **11:59 PM, March 15, 2026**
 - File name format: `MazeFinder_<TeamName>.js`
- **Demonstration & Grading:** Conducted during the theory session

Class Definition

Class Name: `MazeFinder`

```
class MazeFinder {
  this.DIR = { STAY: 0, SOUTH: 1, NORTH: 2, WEST: 4, EAST: 8 };
  /**
   * Constructor
   * @param {number} noOfBot - Number of bots (1-3) to navigate simultaneously
   */
  constructor(noOfBot) {
    // Initialize state for all bots
  }

  /**
   * Calculate next moves for all bots
   * @param {Array<{x: number, y: number, dir: number}>} statuses
   *   - x, y: current position of each bot
   *   - dir: bitmask indicating available directions
   * EAST=8 (1000), WEST=4 (0100), NORTH=2 (0010), SOUTH=1 (0001), STAY=0 (0000)
   * @returns {Array<number>} Array of moves [move1, move2, ...] for each bot
  }
}
```

```

    *      Each move is a direction constant: EAST, WEST, NORTH, SOUTH, or STAY
    */
    nextMove(statuses) {
        // Return array of moves for each bot
        // Example: [this.DIR.EAST, this.DIR.NORTH, this.DIR.SOUTH]
    }
}

```

Example Usage

```

const finder = new MazeFinder(2); // 2 bots

// During simulation
const statuses = [
    { x: 0, y: 0, dir: 0b1001 }, // Bot 1 can go EAST or SOUTH
    { x: 5, y: 3, dir: 0b1100 }  // Bot 2 can go WEST or NORTH
];

const moves = finder.nextMove(statuses);
// Expected: [this.DIR.EAST, this.DIR.NORTH] or similar

```

Grading Criteria

1. Ranking (Ranking Score)

Teams are ranked based on:

1. **Number of mazes completed** (highest priority)
2. **Steps taken** (if tied on #1)
3. **Total path length found by the bots** (if tied on #1 and #2)
4. **Total execution time** (if tied on #1, #2, and #3)

$$\text{Rank Score} = 100 - (\text{Rank} - 1) \times 2$$

2. Report & Presentation (Report Score)

Team must write a short report (1–2 pages) covering all aspects of your algorithm and solution approach.

Instructors may ask any team member to explain technical details. Maximum 100 points.

3. Final Score (Final Score)

$$\text{Final Score} = \sqrt{\text{Rank Score} \times \text{Report Score}}$$

(Geometric mean ensures balanced performance)

How Grading Works

1. Your **MazeFinder** code will be executed automatically against official test mazes
2. Results will be ranked by completion status, efficiency metrics, and solution quality
3. Teams will present and discuss their solutions

FAQ

Q: Can I use built-in sorting or pathfinding libraries? A: No external libraries. Standard JavaScript only.

Q: What if my code throws an error? A: Simulation stops, maze marked as failed, team gets 0 steps/time for that maze.

Q: Can team members have different **MazeFinder implementations?** A: No. Submit one class that represents the entire team's solution.

Q: How many test runs can we do? A: Grading uses one official run per team.

Q: What does "3 bots must reach goal simultaneously" mean? A: Yes, ALL bots must reach the goal cell. Simulation ends when last bot reaches goal.

Q: How is execution time measured? A: Total milliseconds from start until all bots reach goal (step time × number of steps). Note: Path length (number of steps) is the ranking criterion, not wall-clock time.

Q: Can I use different strategies for different maze sizes? A: Yes! You can detect maze size in constructor and adapt accordingly.